

Automated QCOW2 Image Build Pipeline

Go + bash | Minimal Linux | Oracle Cloud Infrastructure

The project explained end to end, in simple terms

The project (as written on the resume): Reduced image-build and qualification time 60% as measured by pipeline metrics by automating a QCOW2 build pipeline in Go and bash that assembled minimal Linux images, installed kernel and platform packages, injected cloud-init and OCI configuration, generated initramfs, and hardened SSH and network settings.

What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail
- Section 6 - The complete workflow, step by step
- Section 7 - How the 60% time saving was achieved and measured
- Section 8 - Common build problems and how they were solved
- Section 9 - Quality safeguards and a 60-second explanation

Note: The resume line gives the result and the main steps. This document explains how such a pipeline is typically built on Linux for OCI. Exact choices (for example Oracle Linux with dnf, xfs vs. ext4, or the exact test tools) may differ in your real setup, and all time numbers in the charts are illustrative examples - your real numbers come from your own pipeline metrics.

1. The Big Picture (Simple Summary)

1.1 The one-line version

I wrote an automatic "image factory" in Go and bash that builds a ready-to-boot Linux disk file (QCOW2) for Oracle Cloud from scratch, sets it up correctly and securely, tests it, and publishes it - and it made the whole build-and-test process 60% faster than before.

1.2 Explained with an everyday example

Imagine a company that ships brand-new laptops to thousands of employees. Every laptop must have the same operating system, the same drivers, the same security settings, and the same tools.

- **The old way:** a person installs everything by hand on each laptop, follows a long checklist, waits around, and then tests each laptop one by one. It is slow, and people make mistakes.
- **The new way:** you build one perfect "master copy" of the laptop's disk automatically, check it with a machine, and then copy it onto every laptop.
- **The master copy** is the **image**. In this project it is a **QCOW2 file** - one file that contains a whole computer disk.
- **The automatic factory** that builds the master copy is the **pipeline**. The "manager" of the factory is written in **Go**, and the individual "workers" doing each job are **bash scripts**.

1.3 The resume line, phrase by phrase

Phrase	What it means in simple words
Reduced image-build and qualification time 60%	Making and testing a new image now takes 40% of the old time. Example: 5 hours became 2 hours.
as measured by pipeline metrics	The saving is not a guess. The pipeline records how long each step takes, and before/after numbers were compared.
automating a QCOW2 build pipeline	Replaced manual steps with a program that does everything by itself and produces a QCOW2 disk file.
in Go and bash	Go is the "manager" program (controls order, timing, errors, parallel work). Bash scripts are the "hands" that run Linux commands.
assembled minimal Linux images	Built a small operating system containing only what is needed - faster, smaller, and safer.
installed kernel and platform packages	Added the core of Linux (the kernel) plus the drivers and tools Oracle Cloud machines need.

Phrase	What it means in simple words
injected cloud-init and OCI configuration	Added the settings that let each new cloud machine set itself up automatically on first start (SSH keys, network, time).
generated initramfs	Built the small "starter kit" the kernel uses to find and open the main disk while booting.
hardened SSH and network settings	Locked down remote login and network behavior so the image is secure by default.

1.4 Why this matters

- **Faster security fixes:** when a security patch comes out, a new tested image is ready in a fraction of the time.
- **Same result every time:** a program does not skip steps or make typos, so every image is identical.
- **Smaller and safer images:** a minimal OS has fewer programs, so fewer things can break or be attacked.
- **Problems are caught early:** cheap, quick checks run first, so bad images fail in minutes instead of hours.
- **Proof with numbers:** stage timers show exactly where time goes, which makes further improvement easy.

2. Key Words in Plain English

Read this table once. Every technical word used later is explained here.

Word	Simple meaning	Example
Image	A saved copy of a whole computer disk, ready to start.	oraclelinux9-2026.09.qcow2
QCOW2	"QEMU Copy-On-Write v2" - a disk-image file format. It only stores used space, supports compression, and is accepted by OCI for import.	A 10 GB disk stored as a 1 GB file.
Pipeline	A chain of automatic steps that run one after another.	Build, check, test, publish.
Go	A programming language that is fast, good at running many tasks at once, and compiles to a single program file.	The orchestrator binary.
bash	The Linux command-line language, ideal for running system commands.	A script that runs mkfs and mount.
Orchestrator	The "manager" program that decides what runs, in what order, and what to do if something fails.	The Go program.
Minimal Linux	Linux with only the essential packages - no desktop, no extra tools.	About 300 packages instead of 1,500.
Root filesystem (rootfs)	All the folders and files of the OS: /etc, /usr, /var, and so on.	The main partition.
chroot	Runs commands "as if" inside the new OS folder, without booting it.	chroot /mnt/root dnf install ...
Kernel	The core of Linux that talks to hardware.	Oracle UEK kernel
Platform packages	Software the cloud platform needs: drivers, agents, time sync, storage tools.	Oracle Cloud Agent, iscsi tools
Driver / module	A small piece of kernel code to use one type of hardware.	virtio_net, nvme
virtio	Fast virtual devices used by virtual machines.	virtio_blk virtual disk
iSCSI	A way to use a disk over the network. OCI bare-metal machines boot from network disks this way.	iscsid service
cloud-init	A program in the image that sets up a new cloud machine on its first boot.	Adds your SSH key, sets hostname.
Datasource	Where cloud-init reads its settings from. On OCI it is the "Oracle" datasource.	datasource_list: [Oracle]

Word	Simple meaning	Example
Metadata service	A special local web address inside the cloud that tells a VM about itself.	169.254.169.254
initramfs	A tiny temporary filesystem loaded with the kernel. It contains just enough drivers to find and mount the real disk.	initramfs-5.15.x.img
dracut	The tool that builds the initramfs on Oracle Linux / RHEL-type systems.	dracut -f
GRUB2	The boot loader - the first program that starts and then loads the kernel.	grub.cfg
BIOS / UEFI	The firmware that starts the computer. UEFI is the modern one and needs a special EFI partition.	EFI System Partition (ESP)
Partition / GPT	Dividing a disk into sections. GPT is the modern partition table.	EFI, /boot, / (root)
qemu-img	Tool to create, convert, and compress disk images.	qemu-img create -f qcow2
qemu-nbd / NBD	Makes a QCOW2 file appear as a normal disk device so Linux can work on it.	/dev/nbd0
SSH	Secure remote login to a Linux machine.	ssh opc@10.0.0.5
Hardening	Changing default settings to make a system harder to attack.	Disable password login.
sysctl	Kernel settings, including many network safety settings.	net.ipv4.tcp_syncookies = 1
SELinux / firewalld	Linux security layers: SELinux limits what programs can do; firewalld blocks unwanted network traffic.	SELinux enforcing
Qualification	Testing that proves an image is good enough to release.	Boot test, SSH test, network test
Pipeline metrics	Numbers the pipeline records about itself: how long each stage takes, pass/fail, retries.	Stage C3 took 6 min 10 s.
Checksum (SHA-256)	A unique fingerprint of a file used to prove it was not changed.	sha256sum image.qcow2
Idempotent	Running a step twice gives the same result as running it once - safe to retry.	mkdir -p

3. Architecture Diagram

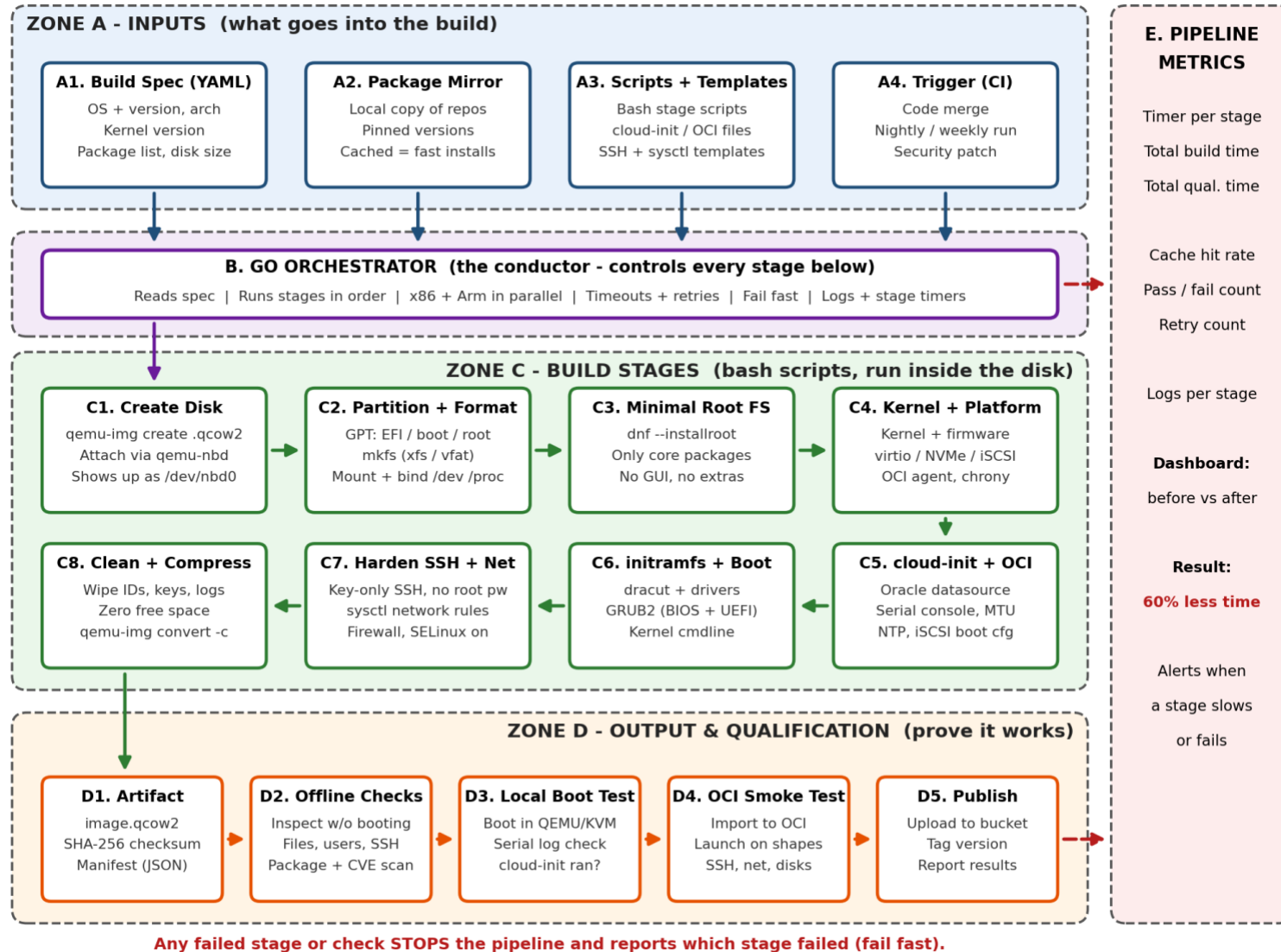


Figure 1 - End-to-end architecture of the QCOW2 build pipeline. Labels (A1-E) match Sections 4 and 5.

4. The Architecture Diagram Explained, Piece by Piece

4.1 How to read the diagram

Visual element	What it means
Big dashed boxes (zones A-E)	Groups of parts with the same job: inputs, control, building, testing, measuring.
Small solid boxes	Individual components. Their labels (A1, B, C1 ... D5, E) match the detailed explanations in Section 5.
Blue	Zone A - Inputs: what the pipeline needs before it can start.
Purple	Zone B - The Go orchestrator: the manager that runs everything.
Green	Zone C - Build stages: bash scripts that actually make the image.
Orange	Zone D - Output and qualification: the finished file and the tests that prove it works.
Red side bar	Zone E - Pipeline metrics: timers and counters that measure every stage (this is how the 60% was proven).
Solid arrows	The order in which work happens.
Red dashed arrows	Measurement data flowing into the metrics system.
Snake layout (C1-C4 left to right, then C5-C8 right to left)	Only a layout choice so arrows never cross. The stages always run in order C1, C2, ... C8.
Red text at the bottom	The fail-fast rule: any failure stops everything immediately.

4.2 Zone A - Inputs

- **A1 Build Spec:** a short YAML file that says what to build (OS version, CPU type, kernel, packages, disk size).
- **A2 Package Mirror:** a local copy of the software repositories, so packages download quickly and always at the same versions.
- **A3 Scripts + Templates:** the bash stage scripts and the configuration files (cloud-init, SSH, sysctl) that get copied into the image.
- **A4 Trigger:** what starts a build - a code change, a schedule, or an urgent security patch.
- **Four blue down-arrows:** all four inputs are handed to the Go orchestrator.

4.3 Zone B - The Go orchestrator

One long box because it controls every stage below it. It reads the spec, runs the stages in order, builds x86 and Arm images at the same time, applies timeouts and retries, stops on the

first failure, and records logs and timers. The **purple arrow** down to C1 shows it starting the build. The **red dashed arrow** to the right sends its timers to the metrics zone.

4.4 Zone C - Build stages

- **C1 Create Disk:** make an empty QCOW2 file and attach it as a disk device.
- **C2 Partition + Format:** divide the disk into EFI, boot, and root sections and format them.
- **C3 Minimal Root FS:** install only the core Linux packages into the new disk.
- **C4 Kernel + Platform:** install the kernel, drivers, and OCI tools.
- **Down arrow C4 to C5:** continue on the second row.
- **C5 cloud-init + OCI:** add the settings that let each cloud VM configure itself on first boot.
- **C6 initramfs + Boot:** build the initramfs and install the boot loader.
- **C7 Harden SSH + Net:** apply security settings.
- **C8 Clean + Compress:** remove anything unique or temporary and shrink the file.
- **Green arrows:** each stage only starts after the previous one succeeds.

4.5 Zone D - Output and qualification

- **Down arrow C8 to D1:** the finished image leaves the build area.
- **D1 Artifact:** the QCOW2 file, its checksum, and a manifest describing what is inside.
- **D2 Offline Checks:** inspect the image without starting it (seconds to minutes).
- **D3 Local Boot Test:** start the image in a local virtual machine and check it boots (minutes).
- **D4 OCI Smoke Test:** import it into Oracle Cloud and start real VMs on several machine types (the slowest test, so it runs last and in parallel).
- **D5 Publish:** upload the passing image, tag its version, and report the results.
- **Orange arrows:** tests go from cheapest to most expensive, so most problems are found early.

4.6 Zone E - Pipeline metrics

Collects a timer for every stage, total build and qualification time, cache hit rate, pass/fail counts, retries, and logs. A dashboard compares before and after - which is where the **60% less time** result comes from. It can also alert the team when a stage slows down or fails.

4.7 Every arrow in one table

From	To	What travels along the arrow
A1-A4	B	Build spec, package source, scripts, and the start signal.
B	C1	The "start building" command, with settings from the spec.

From	To	What travels along the arrow
C1 to C8	Next stage	A disk that has been improved by one more step.
C8	D1	The finished, compressed QCOW2 file.
D1 to D5	Next check	The same image, now trusted a little more after each passing test.
B	E	Stage start/end times, retries, errors (red dashed).
D5	E	Test results and total pipeline time (red dashed).

5. Every Component Explained in Detail

Each component has four parts: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. Some also show a short example of the real commands or code (in grey boxes). Labels match the diagram.

A1 - Build Spec (the order form)

In simple terms

A small YAML text file that describes exactly what image to build. Change the file, and you get a different image - no code changes needed.

Detailed breakdown

- **OS and version:** for example Oracle Linux 9.4.
- **Architecture:** x86_64 or aarch64 (Arm).
- **Kernel:** exact kernel package and version to install.
- **Package list:** the minimal set plus platform packages, with pinned versions.
- **Disk layout:** total size and partition sizes.
- **Options:** firmware type (BIOS, UEFI or both), which tests to run, output name.
- **Validation:** the Go program checks the spec before starting, so a typo fails in one second, not after an hour.

Example

```
name: ol9-minimal
arch: x86_64
kernel: kernel-uek-5.15.0
disk_size: 10G
firmware: [bios, uefi]
packages: [ "@core", cloud-init, oracle-cloud-agent ]
```

Why it matters

One clear source of truth. Anyone can see exactly what is in an image.

Everyday analogy: An order form at a sandwich shop: bread, filling, extras - written down, so the result is always what was ordered.

A2 - Package Mirror / Cache (the local pantry)

In simple terms

A copy of the software repositories kept close to the build machines.

Detailed breakdown

- **Speed:** downloading from a nearby mirror is much faster than from the internet.
- **Pinned versions:** the mirror is a snapshot, so the same build gives the same packages.
- **Reliability:** builds do not fail because an external server is slow or down.

- **Cache reuse:** packages downloaded once are reused by later builds.
- **Big part of the 60%:** package download time drops sharply.

Why it matters

Removes slow, unpredictable downloads - one of the biggest time savings.

Everyday analogy: Keeping ingredients in your own pantry instead of driving to the store each time you cook.

A3 - Scripts + Templates (the instruction cards)

In simple terms

The bash stage scripts and the ready-made config files the pipeline copies into the image, kept in Git.

Detailed breakdown

- **One script per stage:** 10-create-disk.sh, 20-partition.sh, ... 80-cleanup.sh.
- **Config templates:** cloud-init files, sshd_config, sysctl files, chrony.conf, GRUB defaults.
- **Version control:** every change is reviewed and can be traced back.
- **Shared helper library:** common functions (logging, retry, mount helpers) in one file sourced by all scripts.

Why it matters

Clear separation of jobs makes each step easy to test, fix, and time.

Everyday analogy: A deck of instruction cards, one card per task.

A4 - Trigger (the start button)

In simple terms

What makes a build start.

Detailed breakdown

- **Code change:** merging a change to the spec or scripts.
- **Schedule:** for example nightly or weekly builds to pick up new patches.
- **Urgent patch:** a manual start when a critical security fix is released.
- **CI system:** the CI tool runs the Go binary on a build machine with the right permissions.

Why it matters

Builds happen automatically, without someone remembering to start them.

Everyday analogy: An alarm clock that starts the factory.

B - Go Orchestrator (the manager)

In simple terms

A Go program that reads the spec and runs every bash stage in the right order, watches the time, handles errors, runs things in parallel, and records everything.

Detailed breakdown

- **Command-line tool:** one binary, for example `imagebuilder build --spec ol9.yaml`.
- **Parses the spec** into Go structures and validates it (required fields, sizes, allowed values).
- **Stage list:** each stage has a name, a script, a timeout, and a retry rule.
- **Runs bash safely:** uses `exec.CommandContext` so a stuck stage is killed after its timeout.
- **Passes settings** to scripts through environment variables (`ROOT_DIR`, `DEVICE`, `KVER`, `ARCH`).
- **Parallel builds:** goroutines build x86 and Arm images at the same time; parallel OCI tests run the same way.
- **Retries with backoff:** only for steps that can fail for temporary reasons (downloads, cloud API calls).
- **Fail fast:** the first real error stops the run and reports the stage name and log.
- **Always cleans up:** unmounts filesystems and disconnects the disk device even when a stage fails (Go `defer`), so the next build starts clean.
- **Structured logs:** every line is tagged with build ID and stage.
- **Timers:** records start and end time of every stage and sends them to the metrics system.

Example

```
ctx, cancel := context.WithTimeout(parent, stage.Timeout)
defer cancel()
cmd := exec.CommandContext(ctx, "bash", stage.Script)
cmd.Env = append(os.Environ(), "ROOT_DIR="+root, "ARCH="+arch)
start := time.Now(); err := cmd.Run()
metrics.Record(stage.Name, time.Since(start), err)
```

Why it matters

Bash is great at running commands but weak at timeouts, parallel work, and clean error handling. Go adds these, which makes the pipeline both faster and more reliable.

Everyday analogy: A project manager who hands out tasks, keeps a stopwatch, and stops the line when something goes wrong.

Bash stage script conventions (how every worker behaves)

In simple terms

Rules every bash script follows so it is safe and predictable.

Detailed breakdown

- **set -euo pipefail:** stop on any error, on unset variables, and on failures inside pipes.
- **trap cleanup EXIT:** always undo mounts or temporary files when the script ends.
- **Idempotent steps:** safe to run again (mkdir -p, check before creating).
- **No secrets:** scripts never write passwords or keys into the image.
- **Clear logging:** each script prints what it is doing, which appears in the stage log.

Example

```
#!/usr/bin/env bash
set -euo pipefail
trap 'umount -R "$ROOT_DIR" || true' EXIT
```

Why it matters

A small mistake stops the build immediately instead of silently producing a broken image.

Everyday analogy: Workplace safety rules every worker follows.

C1 - Create Disk

In simple terms

Creates an empty QCOW2 file and connects it so Linux treats it like a real disk.

Detailed breakdown

- **qemu-img create:** makes an empty QCOW2 file of the chosen size. It is tiny at first because QCOW2 only stores used space.
- **Load NBD:** the nbd kernel module lets Linux attach image files as disks.
- **qemu-nbd --connect:** the file appears as /dev/nbd0.
- **Lock:** the orchestrator picks a free NBD device so parallel builds do not clash.

Example

```
qemu-img create -f qcow2 disk.qcow2 10G
modprobe nbd max_part=8
qemu-nbd --connect=/dev/nbd0 disk.qcow2
```

Why it matters

Building directly into the image file is faster than building a full VM and snapshotting it.

Everyday analogy: Taking a new blank hard drive out of the box and plugging it in.

C2 - Partition + Format

In simple terms

Divides the blank disk into sections and prepares each section to hold files.

Detailed breakdown

- **GPT partition table** (modern layout).
- **Partition 1 - EFI System Partition** (~200 MB, FAT32) for UEFI boot.

- **Partition 2 - /boot** (~1 GB, xfs) for the kernel and initramfs.
- **Partition 3 - / (root)** (the rest, xfs) for the OS.
- **BIOS boot support** (a tiny BIOS-boot partition) if the image must boot on both BIOS and UEFI.
- **Format** each partition and **mount** them under a build folder.
- **Bind-mount /dev, /proc, /sys** so commands inside the chroot work.
- **Labels / UUIDs** are recorded so fstab and GRUB can find the partitions.

Example

```
parted -s /dev/nbd0 mklabel gpt
mkfs.vfat -F32 /dev/nbd0p1; mkfs.xfs /dev/nbd0p2; mkfs.xfs /dev/nbd0p3
mount /dev/nbd0p3 $ROOT_DIR
```

Why it matters

The right layout lets the same image boot on many OCI machine types.

Everyday analogy: Dividing a new house into rooms before moving in furniture.

C3 - Minimal Root Filesystem

In simple terms

Installs a small, clean Linux into the root partition - only what is needed.

Detailed breakdown

- **dnf --installroot:** installs packages into the new disk folder instead of the build machine.
- **Minimal group** (for example @core) and **no weak dependencies**, to avoid extras.
- **No desktop, no compilers, no documentation** unless the spec asks for them.
- **Uses the package mirror (A2)** for speed and fixed versions.
- **Basic files:** fstab using UUIDs, locale, timezone (UTC), hostname placeholder.
- **Result:** a smaller image that builds, uploads, and boots faster and has less to patch.

Example

```
dnf --installroot=$ROOT_DIR --releasever=9 -y \
  --setopt=install_weak_deps=False install @core
```

Why it matters

Fewer packages means less build time, a smaller file, and fewer security risks.

Everyday analogy: Packing only what you need for a trip - the bag is lighter and faster to carry.

C4 - Kernel + Platform Packages

In simple terms

Adds the Linux kernel and all the drivers and tools that Oracle Cloud machines need.

Detailed breakdown

- **Kernel:** the exact version from the spec (for example Oracle UEK) plus firmware packages.
- **Virtual devices:** virtio disk and network drivers for VMs.
- **Storage:** NVMe support and iSCSI tools (iscsi-initiator-utils) for bare metal and network boot volumes.
- **OCI tools:** Oracle Cloud Agent (monitoring, management plugins) and OCI utilities.
- **Time:** chrony for accurate time.
- **Must-haves:** cloud-init, openssh-server, firewalld, SELinux policy packages.
- **Architecture-aware:** the Arm build gets Arm packages from the same spec.
- **Kernel version is saved** (KVER) and passed to C6 so initramfs is built for the right kernel.

Why it matters

Without the right drivers and agents, the image might not boot or might not work with OCI features.

Everyday analogy: Installing the engine and the dashboard electronics into a car body.

C5 - cloud-init + OCI Configuration

In simple terms

Adds the settings that let each new VM set itself up on its first boot in Oracle Cloud.

Detailed breakdown

- **Datasource = Oracle:** cloud-init reads the VM's details from the OCI metadata service (169.254.169.254).
- **Default user:** the standard cloud user (for example opc) gets the customer's SSH key; no password.
- **Network:** DHCP on the primary interface; MTU 9000 (OCI's standard jumbo frames).
- **Time sync:** chrony points to OCI's time server at 169.254.169.254.
- **Serial console:** kernel output goes to ttyS0 so OCI's console connection can show boot messages.
- **iSCSI settings** for shapes that boot from network disks.
- **Disk growth:** the root partition grows automatically to the size the customer chose.
- **Services enabled:** cloud-init, sshd, chronyd, Oracle Cloud Agent start on boot.

Example

```
# /etc/cloud/cloud.cfg.d/99-oci.cfg
datasource_list: [ Oracle ]
system_info:
  default_user: { name: opc }
```

Why it matters

The same image can be launched thousands of times, and each copy personalizes itself correctly.

Everyday analogy: A new phone that asks for your account on first start and sets itself up.

C6 - initramfs + Boot Loader

In simple terms

Builds the small starter kit the kernel needs to find the main disk, and installs the boot loader.

Detailed breakdown

- **Why initramfs is needed:** at boot, the kernel cannot read the root disk until it has the right drivers. The initramfs carries those drivers.
- **dracut** builds it inside the chroot for the exact kernel version from C4.
- **Drivers added:** virtio_blk, virtio_scsi, virtio_net, nvme, iSCSI modules, xfs.
- **No host-only mode:** the initramfs must work on many machine types, not only the build machine.
- **GRUB2** installed for BIOS and/or UEFI; grub.cfg generated.
- **Kernel command line:** serial console (console=ttyS0), network naming and other OCI-recommended options.
- **Check:** lsinitrd confirms the needed drivers are really inside.

Example

```
chroot $ROOT_DIR dracut -f --no-hostonly --kver $KVER \
  --add-drivers "virtio_blk virtio_scsi nvme"
chroot $ROOT_DIR grub2-mkconfig -o /boot/grub2/grub.cfg
```

Why it matters

A wrong or missing initramfs is the most common reason an image does not boot.

Everyday analogy: A small toolkit you carry to open the main toolbox.

C7 - Harden SSH + Network

In simple terms

Changes default settings so the image is secure from the first second it runs.

Detailed breakdown

- **SSH:** PermitRootLogin no, PasswordAuthentication no (keys only), limited login attempts, strong modern ciphers only, idle timeout.
- **Network (sysctl):** ignore ICMP redirects, do not send redirects, reverse-path filtering on, SYN-flood protection on, IP forwarding off, ignore source-routed packets.
- **Firewall:** firewalld enabled, only SSH open by default.
- **SELinux:** enforcing mode.

- **Services:** disable anything not needed.
- **Accounts:** no default passwords; locked system accounts.
- **Compliance:** settings follow a benchmark such as CIS.

Example

```
# /etc/ssh/sshd_config.d/50-hardening.conf
PermitRootLogin no
PasswordAuthentication no
# /etc/sysctl.d/90-net-hardening.conf
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.tcp_syncookies = 1
```

Why it matters

Every VM made from this image inherits these protections automatically.

Everyday analogy: Changing the locks and installing an alarm before anyone moves in.

C8 - Clean + Compress

In simple terms

Removes anything unique or temporary and shrinks the file.

Detailed breakdown

- **Make it generic:** empty /etc/machine-id, delete SSH host keys (each VM makes new ones), run cloud-init clean.
- **Remove leftovers:** logs, package cache, shell history, temporary files.
- **SELinux relabel** so file security labels are correct.
- **Unmount everything** and disconnect /dev/nbd0.
- **Shrink:** fill unused space with zeros (or trim), then qemu-img convert with compression - the output file becomes much smaller.

Example

```
truncate -s 0 $ROOT_DIR/etc/machine-id
rm -f $ROOT_DIR/etc/ssh/ssh_host_*
qemu-img convert -c -O qcow2 disk.qcow2 final.qcow2
```

Why it matters

Prevents all VMs from sharing the same identity or keys, and makes uploads faster.

Everyday analogy: Cleaning a rental apartment and removing the last tenant's name from the mailbox.

D1 - Artifact (the finished product)

In simple terms

The final QCOW2 file plus two helper files.

Detailed breakdown

- **final.qcow2:** the image.
- **SHA-256 checksum:** proves the file was not changed later.
- **Manifest (JSON):** image version, OS, kernel, package list with versions, git commit, build ID, build time.
- **Clear name** including OS, arch, and version.

Why it matters

Anyone can later see exactly what is inside an image, and verify it.

Everyday analogy: A product in a sealed box with a packing list.

D2 - Offline Checks (inspect without starting)

In simple terms

Looks inside the image file without booting it. Very fast.

Detailed breakdown

- **Tools:** libguestfs tools such as virt-inspector or guestfish in read-only mode.
- **Checks:** kernel and initramfs present; initramfs contains the needed drivers; fstab UUIDs match; sshd settings are hardened; no SSH host keys; machine-id empty; no unexpected users or passwords; cloud-init datasource set to Oracle.
- **Package and CVE scan** against the manifest.
- **Takes seconds to a few minutes** and catches many mistakes before any slow test.

Why it matters

Finds problems in minutes instead of waiting for a cloud test to fail.

Everyday analogy: Checking a suitcase with an X-ray scanner before the flight.

D3 - Local Boot Test

In simple terms

Starts the image in a local virtual machine on the build server.

Detailed breakdown

- **QEMU/KVM** boots a throwaway copy of the image.
- **Serial log** is saved and searched for errors (kernel panic, failed services).
- **cloud-init test:** a small local seed provides fake metadata, and the test checks cloud-init finished.
- **SSH login** works with a test key; password login is refused.
- **Both BIOS and UEFI** boot paths are tested.
- **Takes a few minutes** - much faster than a cloud launch.

Example

```
qemu-system-x86_64 -enable-kvm -m 2048 -nographic \
-drive file=test.qcow2,if=virtio -serial file:serial.log
```

Why it matters

Proves the image really boots before paying the time cost of cloud tests.

Everyday analogy: Starting a car in the garage before taking it on the road.

D4 - OCI Smoke Test

In simple terms

Imports the image into Oracle Cloud and launches real VMs on several machine types.

Detailed breakdown

- **Upload** to Object Storage and **import** as a custom image (QCOW2 is a supported format).
- **Launch** on a small set of shapes (for example AMD, Intel, Arm) - in parallel, driven by Go.
- **Checks:** VM reaches RUNNING, SSH works, correct MTU, time synced, block volume attaches, Oracle Cloud Agent reports, serial console shows a clean boot.
- **Cleanup:** terminate test VMs and delete temporary resources automatically.

Why it matters

The final proof that the image works in the real cloud.

Everyday analogy: A test drive on real roads.

D5 - Publish

In simple terms

Makes the passing image available and records the result.

Detailed breakdown

- **Upload** the final QCOW2, checksum, and manifest to the release bucket.
- **Tag** the version (for example ol9-2026.09.22).
- **Report:** a summary with test results and stage timings.
- **Notify** the team.
- **Hand-off** to the next process (for example wider validation and staged rollout).

Why it matters

Only tested images are released, and each one is traceable.

Everyday analogy: Putting the approved product on the store shelf with a label.

E - Pipeline Metrics (the stopwatch and scoreboard)

In simple terms

Records how long every stage takes and whether it passed. This is the evidence for the 60% improvement.

Detailed breakdown

- **Per-stage timers:** start, end, duration for C1-C8 and D1-D5.
- **Totals:** full build time, full qualification time, end-to-end time.
- **Counters:** pass/fail, retries, cache hits vs. misses.
- **Stored per run** with build ID and version, so runs can be compared.
- **Dashboard:** trend over time and before vs. after (use median, and p90, over many runs - not a single lucky run).
- **Alerts:** when a stage becomes slower than normal or fails repeatedly.

Why it matters

You can only improve what you measure. The timers showed where time was wasted and proved the saving.

Everyday analogy: A stopwatch and scoreboard for a relay race.

6. The Complete Workflow, Step by Step

This follows one image from the moment a build is triggered to the moment it is published.

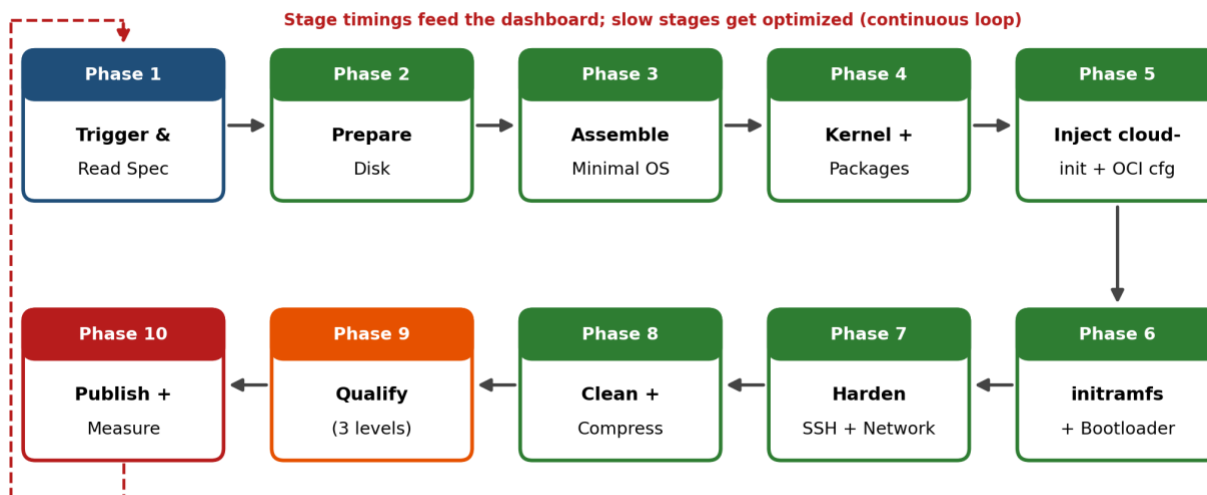


Figure 2 - The 10 phases. Blue = start, green = build, orange = test, red = publish and measure.

Phase 1 - Trigger and read the spec

Step	What happens (detail)	In simple words
1.1	A code merge, schedule, or urgent patch starts the CI job.	Someone (or a clock) presses start.
1.2	CI runs the Go binary with the spec file.	The manager arrives.
1.3	Go parses and validates the spec; bad input fails in seconds.	Check the order form.
1.4	A build ID is created; timers start; a clean work folder is prepared.	Start the stopwatch.
1.5	If several architectures are requested, one build per architecture starts in parallel.	Two production lines at once.

Phase 2 - Prepare the disk

Step	What happens (detail)	In simple words
2.1	Create an empty QCOW2 file of the requested size.	Get a blank disk.
2.2	Attach it with qemu-nbd so it appears as /dev/nbdX.	Plug it in.
2.3	Create GPT partitions: EFI, /boot, root (plus BIOS-boot if needed).	Divide into rooms.

Step	What happens (detail)	In simple words
2.4	Format partitions and mount them in the work folder.	Make rooms usable.
2.5	Bind-mount /dev, /proc, /sys for chroot use.	Connect the utilities.

Phase 3 - Assemble the minimal OS

Step	What happens (detail)	In simple words
3.1	Point dnf to the local package mirror.	Use the nearby pantry.
3.2	Install the minimal package group into the root folder.	Put in only the essentials.
3.3	Write fstab with UUIDs, set locale and timezone.	Basic house rules.

Phase 4 - Install kernel and platform packages

Step	What happens (detail)	In simple words
4.1	Install the exact kernel version and firmware.	Install the engine.
4.2	Install drivers and tools: virtio, NVMe, iSCSI, chrony.	Add the right parts for cloud hardware.
4.3	Install OCI software: Oracle Cloud Agent, OCI utilities, cloud-init.	Add the cloud's own tools.
4.4	Save the kernel version for the initramfs stage.	Note which engine was installed.

Phase 5 - Inject cloud-init and OCI configuration

Step	What happens (detail)	In simple words
5.1	Write cloud-init config with the Oracle datasource and default user.	Teach it how to set itself up.
5.2	Configure networking (DHCP, MTU 9000) and time (OCI NTP).	Network and clock settings.
5.3	Enable serial console output and iSCSI settings.	Make boot messages visible; support network disks.
5.4	Enable services to start at boot.	Turn on the right switches.

Phase 6 - Generate initramfs and install the boot loader

Step	What happens (detail)	In simple words
6.1	Run dracut in chroot for the saved kernel version, with cloud drivers and no host-only mode.	Pack the starter toolkit.
6.2	Verify the initramfs contains the needed drivers.	Check the toolkit.
6.3	Install GRUB2 for BIOS/UEFI and generate grub.cfg with the right kernel options.	Install the ignition key.

Phase 7 - Harden SSH and network

Step	What happens (detail)	In simple words
7.1	Apply SSH hardening: no root login, keys only, strong ciphers.	Change the locks.
7.2	Apply sysctl network hardening.	Set network safety rules.
7.3	Enable firewall (SSH only) and SELinux enforcing.	Turn on the alarm.
7.4	Disable unneeded services and lock system accounts.	Close unused doors.

Phase 8 - Clean and compress

Step	What happens (detail)	In simple words
8.1	Clear machine-id, SSH host keys, cloud-init state, logs, caches, history.	Remove fingerprints.
8.2	Relabel SELinux, unmount, disconnect NBD.	Close up the box.
8.3	Zero or trim free space and convert with compression.	Vacuum-pack it.
8.4	Write checksum and manifest.	Seal and label.

Phase 9 - Qualify in three levels (fast to slow)

Step	What happens (detail)	In simple words
9.1	Level 1 - Offline checks with libguestfs tools (seconds to minutes).	X-ray the box.
9.2	Level 2 - Local QEMU/KVM boot test for BIOS and UEFI, cloud-init and SSH checks (minutes).	Start it in the garage.
9.3	Level 3 - Import to OCI and launch on several shapes in parallel; run smoke tests (longest).	Test drive on real roads.

Step	What happens (detail)	In simple words
9.4	Any failure stops the pipeline and reports the failing stage and log.	Stop the line.
9.5	Clean up all test VMs and temporary cloud resources.	Tidy up.

Phase 10 - Publish and measure

Step	What happens (detail)	In simple words
10.1	Upload image, checksum, and manifest to the release bucket; tag the version.	Put it on the shelf.
10.2	Send stage timings and results to the metrics system.	Record the lap times.
10.3	Dashboard compares with previous runs; alerts on slow or failing stages.	Check the scoreboard.
10.4	Slowest stages are reviewed and optimized in the next iteration.	Keep improving.

7. How the 60% Time Saving Was Achieved and Measured

7.1 The idea in one sentence

Stop doing slow things by hand and one at a time; do them automatically, in parallel, with local caches, and run cheap tests before expensive ones.

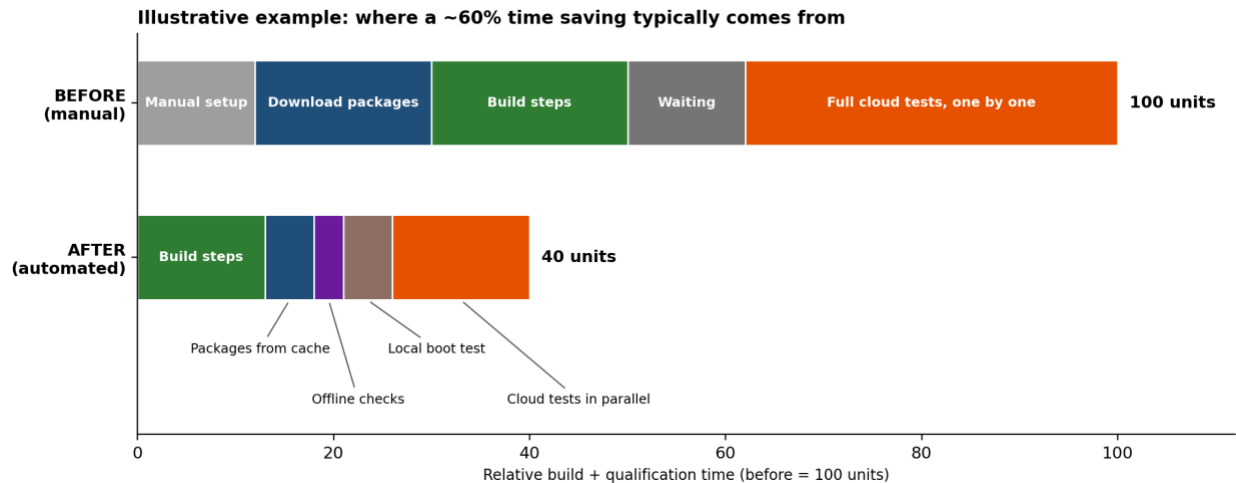


Figure 3 - Illustrative only. Real per-stage numbers come from your own pipeline metrics.

7.2 Where the time was saved

Technique	Before	After	Why it saves time
Full automation (Go + bash)	Engineer runs steps by hand	One command runs everything	No waiting for people, no hand-offs, no forgotten steps.
Build straight into a QCOW2 file	Launch a VM, set it up, snapshot it	Build offline into the disk file	No VM boot/shutdown cycles during the build.
Minimal package set	Large default install	Only essential packages	Less to download, install, compress, upload, and scan.
Local package mirror and cache	Download from internet each time	Nearby cached, pinned repos	Downloads become much faster and more reliable.
Parallel work (Go goroutines)	x86, Arm, and tests one after another	Run at the same time	Total time is the longest task, not the sum of tasks.
Tiered qualification	Every check done in the cloud	Offline, then local boot, then cloud	Most bugs are caught in minutes; slow cloud tests run only on good images.
Fail fast	Problems found at the end	Stop at the first error	No time wasted finishing a build that is already broken.

Technique	Before	After	Why it saves time
Compression	Large image file	Compressed, zeroed free space	Faster upload and import into OCI.
Automatic cleanup and retries	Manual fixes for stuck mounts and flaky downloads	defer cleanup, targeted retries	Fewer failed or restarted runs.

7.3 How the 60% was measured

- **Same scope:** compare the same kind of job before and after - build plus qualification of the same OS image.
- **Recorded, not estimated:** every stage writes its own start and end time to the pipeline metrics.
- **Many runs:** use the median (and p90) over many runs, so one lucky or unlucky run does not decide the number.
- **Formula:** saving = (old time - new time) / old time. Example: (300 min - 120 min) / 300 min = 60%.
- **Stage view:** the per-stage breakdown shows exactly which improvements contributed the most.

8. Common Build Problems and How They Were Solved

Problem	Likely cause	How it was found	Fix
Image does not boot ("cannot find root device")	initramfs missing virtio/NVMe driver or built in host-only mode	Local boot test serial log; lsinitrd	Build with --no-hostonly and add the drivers explicitly
Boots on BIOS but not UEFI	EFI partition or GRUB EFI files missing	Local UEFI boot test	Create ESP and install EFI boot files
Cannot SSH into new VM	cloud-init datasource wrong or SSH key not injected	Serial console; cloud-init logs	Set Oracle datasource; check default user config
All VMs have the same SSH host key	Keys not removed during cleanup	Offline check	Delete host keys in C8; add check in D2
Slow or broken network	Wrong MTU or interface naming	OCI smoke test	Set MTU 9000 via DHCP; fix naming options
Build fails at random	Flaky package download	Retry counters in metrics	Local mirror plus targeted retries
Next build fails: "device busy"	Earlier run left mounts or /dev/nbd attached	Stage logs	Go defer cleanup + bash trap on every exit
Parallel builds collide	Two builds used the same /dev/nbd device	Logs	Orchestrator reserves a free device per build
Stage suddenly slower	Cache miss or mirror outage	Stage timer alert	Fix mirror; alert on cache hit rate
Time wrong on VMs	chrony not pointed at OCI time server	Smoke test	Configure 169.254.169.254 as NTP source

9. Quality Safeguards and a 60-Second Explanation

9.1 Built-in safeguards

- **Spec validation** before any work starts.
- **set -euo pipefail** in every bash script.
- **Timeouts** on every stage so nothing hangs forever.
- **Fail fast** with the failing stage name and log.
- **Guaranteed cleanup** of mounts and devices.
- **Three-level testing** from cheap to expensive.
- **Checksum and manifest** for every image.
- **No secrets** ever written into an image.
- **Metrics and alerts** on every stage.

9.2 The 60-second explanation (for interviews)

"Building and qualifying our OCI Linux images used to be slow and partly manual. I automated it as a pipeline: a Go orchestrator that reads a YAML spec and runs bash stages with timeouts, retries, parallelism, and guaranteed cleanup. The stages create a QCOW2 disk, partition it, install a minimal Linux from a local mirror, add the kernel and OCI platform packages, inject cloud-init and OCI settings like the Oracle datasource, serial console, MTU and NTP, generate a portable initramfs with dracut, install GRUB for BIOS and UEFI, harden SSH and network settings, then clean and compress the image. Qualification runs from fast to slow: offline inspection, a local KVM boot, then parallel smoke tests on real OCI shapes. Every stage is timed, and pipeline metrics showed build plus qualification time dropped by 60%."